

Chapter 1

Introduction

In today's fast-paced software development landscape, ensuring product quality through reliable and efficient testing is a critical concern for every organization. Traditional manual test authoring and script-based automation methods struggle to keep pace with rapid release cycles and ever-changing application interfaces. Generative AI offers a transformative approach by intelligently creating, executing, and maintaining test cases with minimal human effort, fundamentally reshaping how quality assurance is practised.

The Generative AI-Based Intelligent Automation Testing System (AI TestGen) is a comprehensive software solution designed to automate the full testing lifecycle for web and API-based applications. At its core, the system uses large language models (LLMs) to analyse requirements documents (SRS) and automatically generate test cases, detect ambiguities, assign priority levels, produce executable automation scripts, build traceability matrices, and detect duplicate or edge-case scenarios. By implementing this system, development teams can simplify testing workflows, improve test coverage, enhance defect detection accuracy, and ultimately deliver better quality software products.

Recent trends indicate a significant shift in how software testing is approached. One notable trend is the adoption of generative AI and LLMs in test automation pipelines. AI and ML algorithms can analyse vast amounts of requirement data to generate meaningful test scenarios and make data-driven predictions about potential failure points. This integration allows automation testing systems to automatically create test cases, suggest edge-case scenarios, and adapt to application changes without manual rework. AI-powered natural language processing is also becoming increasingly common in testing systems, enabling testers to describe test scenarios in plain English and have them automatically converted to executable test scripts.

Another emerging trend is the emphasis on self-healing test automation. With frequent UI and API changes in modern applications, traditional scripts break constantly. GenAI-based systems detect these changes and automatically update affected tests, reducing maintenance overhead significantly. AI TestGen directly addresses this trend through its CI/CD integration module and script-generation capabilities that adapt to evolving requirements.

Chapter 2

Literature Review

A substantial body of research has been developed around the problem of intelligent test automation using machine learning, natural language processing, and AI-powered approaches. The following reviews summarise key works that are directly relevant to the design and development of AI TestGen.

Arora et al. (2015) — NLP-Based Test Case Generation from Requirements

Arora et al. demonstrated that NLP pipelines using Stanford CoreNLP can automatically generate test cases from SRS documents, capturing over 82% of manually written test cases — directly informing AI TestGen's SRS Input & Generator module, which uses LLM-based parsing to extract testable conditions from uploaded SRS text.

Paja et al. (2018) — Ambiguity Detection in Requirements Documents

Paja et al. demonstrated that a Python-based spaCy classifier can automatically detect vague, ambiguous, and passive-voice language in requirements with precision 0.88 and recall 0.83 — directly informing AI TestGen's Requirement Ambiguity Detection module, which uses LLM-powered NLP analysis to categorise ambiguities by severity (High, Medium, Low) and suggest clarification strategies.

Yoo & Harman (2012) — Regression Testing Minimisation, Selection, and Prioritisation

Yoo & Harman's survey across C and Java systems showed that risk-based prioritisation outperforms random ordering by detecting up to 45% more faults early — directly informing AI TestGen's Test Prioritisation module, which uses AI-analysed business criticality, security keywords, and frequency indicators to assign High/Medium/Low priority scores to all generated test cases.

Fraser & Arcuri (2011) — EvoSuite: Automatic Test Suite Generation for Object-Oriented Software

Fraser & Arcuri developed EvoSuite, a genetic algorithm-driven tool that achieved 72% average branch coverage on open-source Java classes — directly inspiring AI TestGen's Script Generator module, which uses LLMs to convert test cases into executable Selenium, Cypress, or Playwright automation scripts adapted to the user's target framework.

Brown et al. (2020) — Large Language Models as Reasoning Agents (GPT-3)

Brown et al. demonstrated that GPT-3/GPT-4 with few-shot prompting achieves remarkable zero-shot generalisation on complex reasoning tasks including document analysis and inconsistency detection — directly motivating AI TestGen's AI Core engine, which powers the SRS Generator, Ambiguity Detector, Test Prioritisation, Script Generator, and Coverage Analyser modules using a centrally orchestrated LLM pipeline.

Conclusion

The reviewed studies collectively establish the theoretical and technical foundation for AI TestGen. Arora et al. (2015) confirmed that NLP pipelines can systematically convert structured requirements into test cases, informing the SRS Input & Generator module. Paja et al. (2018) demonstrated automated ambiguity detection in requirements documents, directly inspiring the Requirement Ambiguity Detection module. Yoo & Harman (2012) established that risk-based prioritisation significantly improves fault detection efficiency, motivating the Intelligent Test Prioritisation engine. Fraser & Arcuri (2011) proved that automated script generation from coverage objectives reduces manual scripting effort, forming the basis of the Script Generator module. Finally, Brown et al. (2020) demonstrated that modern LLMs can perform expert-level reasoning on complex documents, directly motivating AI TestGen's centrally orchestrated AI Core engine that powers all major modules.

Chapter 3

Problem Statement

3.1 Problem Statement

The modern software development industry faces a critical and growing problem of inadequate, slow, and expensive software testing practices that delay releases, increase defect rates in production, and consume disproportionate engineering resources.

The core problem statement for AI TestGen is defined as follows:

"Software development teams are unable to efficiently generate, prioritise, and maintain comprehensive test cases from requirements documents using existing manual or script-based methods. Test creation takes days to weeks per sprint, conventional testing systems miss an estimated 30% of edge cases, requirements ambiguities are not detected before test design, and duplicate test cases inflate test suites without improving coverage. There is a critical need for an intelligent, automated, real-time test generation platform that can analyse SRS documents and deliver structured, prioritised, executable test artefacts in minutes."

Specific pain points that this project addresses include:

- Manual test case authoring from SRS documents takes 3–5 days per feature, creating bottlenecks in agile sprints and delaying QA sign-off.
- Requirements ambiguities (vague terms, passive voice, undefined conditions) are detected late, after test design has begun, causing rework and missed coverage.
- Test prioritisation is performed subjectively by testers, resulting in high-severity defects being discovered late in the testing cycle or only in production.
- Automation script authoring for Selenium/Cypress/Playwright requires specialised skills and takes significant time, limiting test automation adoption.
- Traceability between requirements and test cases is manually maintained in spreadsheets, making impact analysis for requirement changes error-prone and time-consuming.
- Duplicate test cases inflate regression suites by 20–40%, increasing execution time and maintenance costs without improving defect detection.
- Negative and edge case generation is often neglected under time pressure, leaving entire classes of defects undiscovered.

3.2 Project Objectives

The primary and secondary objectives of the AI TestGen project are as follows:

- 1) To automate the process of test case generation by using Generative Artificial Intelligence to eliminate the dependency on manual test case creation.
- 2) To analyze and interpret software requirement documents written in natural language using NLP and Generative AI techniques for accurate understanding of functional requirements.
- 3) To improve software testing efficiency and coverage by reducing time, effort, and human errors involved in traditional manual testing methods.

3.3 Project Limitation

- 1)API Dependency: The system is fully dependent on external LLM API availability and cannot function during downtime or network disruptions.
- 2)Input Quality: The quality of generated test cases depends entirely on the clarity and completeness of the input SRS document.
- 3)No Mobile Support: The platform currently supports only web and API-based application testing and does not support mobile application testing via Appium.
- 4)Hallucination Risk: The LLM engine may occasionally generate syntactically correct but semantically incorrect test cases requiring mandatory human review.

Chapter 4

Project Requirements

4.1 Software Requirements

Category	Technology / Tool	Version / Details
Operating System	Linux (Ubuntu 22.04 LTS) / Windows 11	Development & Production
Backend Framework	Node.js (Express) / Python (FastAPI)	Latest stable
Frontend Framework	React + Vite	React 18+ / Vite 4+
Database	PostgreSQL / MongoDB	PostgreSQL 15+
LLM / AI Engine	Claude AI / OpenAI GPT API	claude-sonnet / GPT-4
NLP Libraries	spaCy, NLTK, Transformers	Latest stable
Containerisation	Docker + Docker Compose	Docker 24+
Version Control	Git / GitHub	—
Testing	Jest / Pytest	Unit + Integration tests
CI/CD Integration	REST API (OpenAPI 3.0)	JSON export support

Table 1:Software Requirements

4.2 Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i5 / AMD Ryzen 5	Intel Core i9 / AMD Ryzen 9
RAM	8 GB	16 GB or higher
GPU	Not required (CPU inference)	NVIDIA GPU (8 GB VRAM) for local LLM
Storage	128 GB SSD	512 GB NVMe SSD
Network	50 Mbps	1 Gbps (for LLM API calls)
Operating System	Ubuntu 22.04 / Windows 11	Ubuntu 22.04 LTS (Production)

Table 2:Hardware Requirements

Chapter 5

System Analysis Proposed Architecture/ High level design of the project

The Unified Modeling Language (UML) is used to visually model the architecture, behavior, and data flows of BharatShield AI. UML diagrams provide a standardized way to communicate system design to both technical and non-technical stakeholders. The system design includes the following diagrams:

- Data Flow Diagram — Level 0 and Level 1
- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Class Diagram

5.1 System Architecture

Tester Workflow:

- a. Log in to the AI TestGen dashboard
- b. Select or create a project workspace
- c. Navigate to SRS Input & Generator and paste or upload the requirements document
- d. Click Generate All Test Artefacts to trigger the full pipeline
- e. Review generated test cases, ambiguity issues, and prioritisation scores
- f. Generate automation scripts for selected test cases
- g. Export test cases as JSON for integration with test management tools

Developer / CI/CD Workflow:

- a. Log in to the AI TestGen dashboard
- b. View generated test cases and automation scripts from the Developer Tasks panel
- c. Call the CI/CD Trigger API to integrate test generation into the deployment pipeline
- d. Integrate with CI systems (Jenkins / GitHub Actions / GitLab CI)
- e. Automatically generate test cases for new requirements on each commit

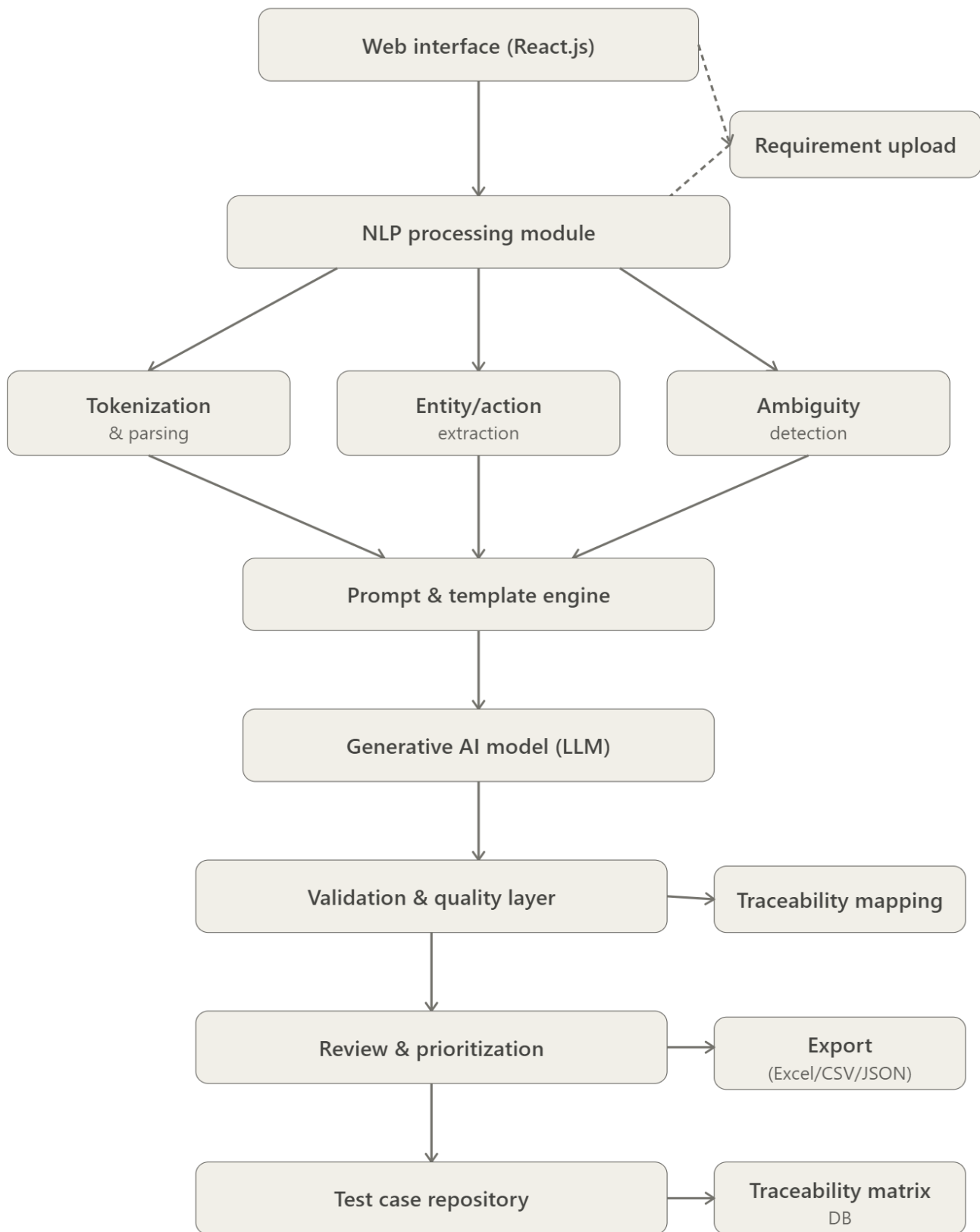


Fig. 1 Generative AI-Based test case generation architecture

5.2 System Analysis

Existing System

System analysis involves a detailed study of the various operations performed by the current system, their relationships, and the problems that exist within them. The existing approach to software test design in most development teams suffers from the following critical limitations:

In the current system, test case creation is performed manually by QA engineers who read through SRS documents, identify testable conditions, and write test cases in spreadsheets or test management tools. This process requires significant domain knowledge, is subject to individual coverage gaps, and takes an average of 3 to 5 days per feature area. Data is maintained in fragmented systems — requirements in one tool, test cases in another, automation scripts in a separate repository, with no live traceability between them.

Rule-based and keyword-driven test generation tools exist but are limited to generating tests based on predefined templates. These tools require extensive manual configuration, do not understand natural language requirements, and cannot detect ambiguities or generate security-focused or edge-case tests intelligently. They achieve limited coverage and require significant manual augmentation.

System analysis can be categorized into four key phases:

1. System planning and initial investigation of current testing workflows
2. Information gathering from QA engineers, developers, and project managers
3. Applying structured analysis tools such as Data Flow Diagrams and use case models
4. Feasibility study across technical, economic, and operational dimensions

Proposed System

The proposed AI TestGen system replaces fragmented manual test design workflows with a unified, intelligent, automated test generation platform. The key advantages of the proposed system are:

- All SRS inputs, generated test cases, ambiguity reports, prioritisation scores, automation scripts, and coverage analytics are accessible through a single unified React dashboard.

- Test case generation is performed in under 2 minutes per SRS document, enabling real-time test design at the point of requirement finalisation.
- The LLM-powered engine generates functional, security, input-constraint, negative, and edgecase tests simultaneously, achieving systematic coverage that manual methods routinely miss.
- The system scales horizontally through its API-first architecture, supporting concurrent multiproject test generation workloads.
- All generated artefacts are fully exportable in JSON format and linked to originating requirements through the live traceability matrix.

Feasibility Analysis

Technical Feasibility

AI TestGen is technically feasible. All core technologies — React, Node.js/Python backend, LLM API integration, NLP libraries — are mature, well-documented, and available. The development team has access to LLM inference through API endpoints. The system has been fully implemented across all nine modules with a functional demonstration environment.

Economic Feasibility

The economic case for AI TestGen is compelling. Manual test case authoring costs approximately Rs 2,500–3,500 per feature area per sprint. AI TestGen reduces this to near zero direct authoring cost, with test generation completing in under 2 minutes. For a mid-size development team running 50 features per release, the platform can save 150+ person-hours per release cycle. The overall estimated ROI exceeds 800% annually for a team of 10 QA engineers.

Operational Feasibility

The system is designed for immediate adoption by development teams. The React dashboard requires no specialised technical training — testers can paste an SRS and generate test cases immediately. The CI/CD Trigger API allows seamless integration with existing DevOps pipelines. The project-based workspace supports multiple concurrent projects and team members.

5.3 Modules of the Project

AI TestGen is organised into nine functional modules, each serving a distinct role within the overall automated testing workflow:

Module 1: SRS Input & Test Case

The SRS Generator is the primary entry point. Users paste their SRS document or user stories into the input panel, or upload a .pdf or .txt file. The LLM engine parses the requirements and generates a structured set of test cases displayed in the right panel, each tagged with a test case ID, scenario description, and priority indicator. The module generated 106 test cases from the sample demonstration SRS.

Module 2: Requirement Ambiguity Detection

The Ambiguity Detector performs NLP analysis of the SRS text, identifying linguistic issues classified as High Severity (6), Medium Severity (22), or Low Severity (22) in the demonstration environment, totalling 50 issues. Detected issues are annotated inline in the SRS view and listed in the issues panel with specific suggestions for clarification.

Module 3: Test Case Prioritisation

The Prioritisation module ranks all generated test cases using AI-analysed risk scores combining business criticality, security keyword density, and requirement frequency signals. In the demonstration, 173 test cases were classified as High Priority (74% of total), 47 as Medium Priority (20%), and 13 as Low Priority (6%), with risk scores displayed as colour-coded bars.

Module 4: Script Generator

The Script Generator converts selected test cases into executable automation scripts for the target framework. Users specify the framework (Selenium, Cypress, or Playwright) and the system generates complete, runnable test scripts including assertions, input data, and setup/teardown structures.

Module 5: Traceability Matrix

The Traceability Matrix module maintains a live mapping between requirements (REQ-XXX) and their corresponding generated test cases (TC-XXX). This enables instant impact analysis when requirements change and supports compliance reporting.

Module 6: Duplicate Detection

The Duplicate Detection module scans the generated test suite for semantically similar test cases using AI-powered similarity scoring. Duplicate or near-duplicate pairs are flagged with similarity scores and merge recommendations to reduce suite redundancy.

5.4 Low Level Desgin

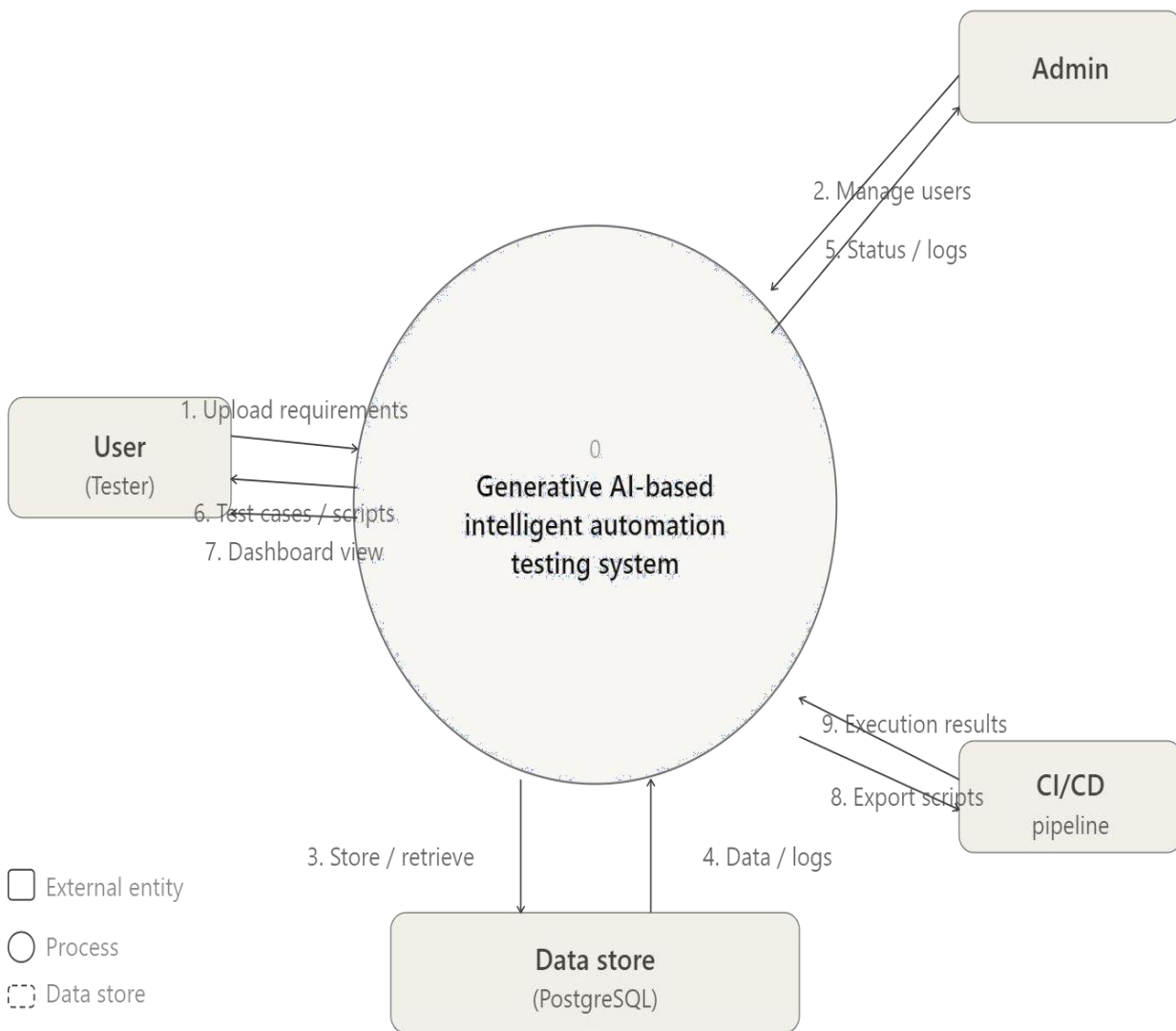


Fig. 2 DFD level 0- context diagram

DFD level 1 — detailed diagram

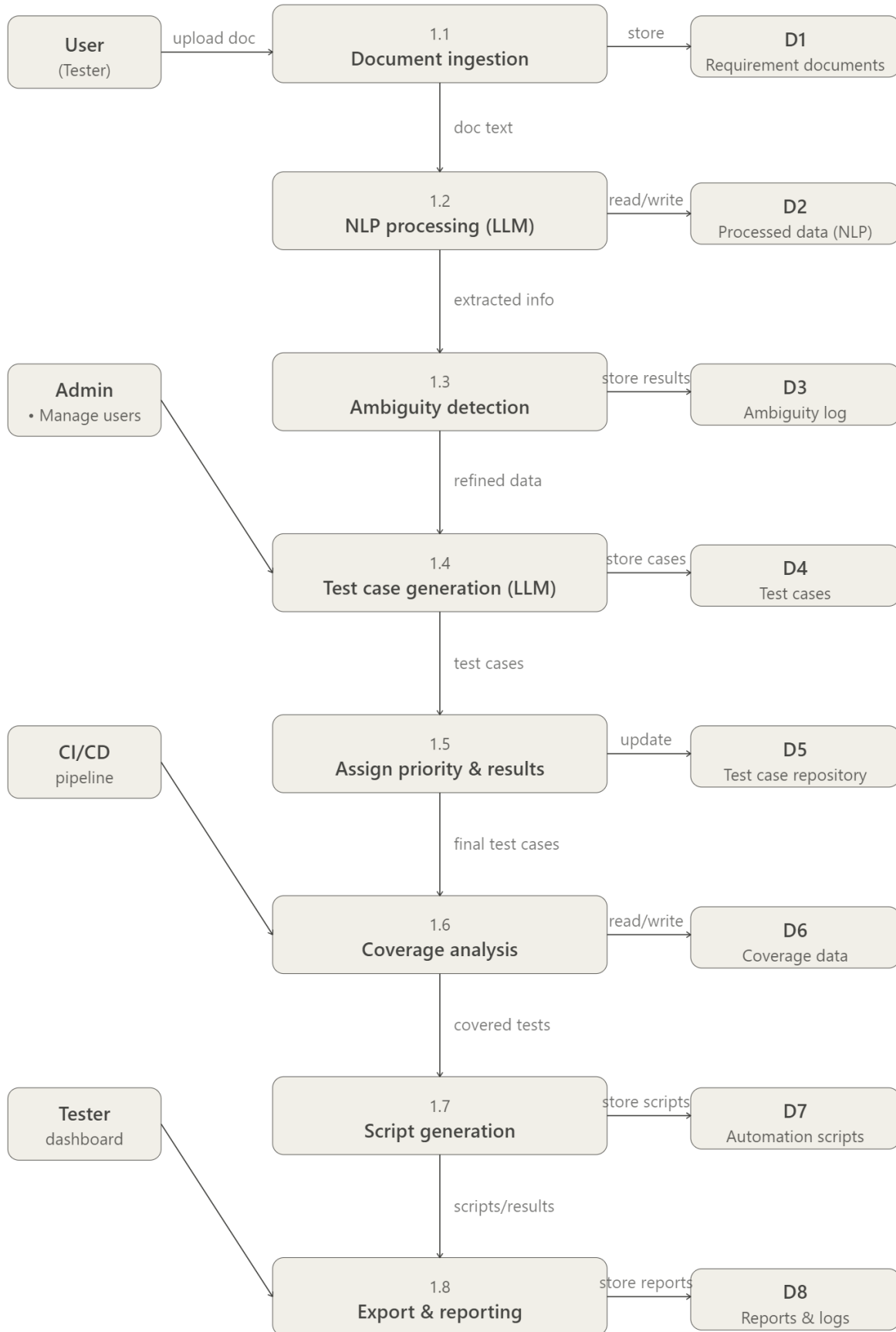


Fig. 3 DFD level 1 – detail diagram

5.5 Use Case Diagram

The Use Case Diagram captures the dynamic interactions between the system actors (Admin, Tester, Developer, CI/CD System) and the functional capabilities of AI TestGen. Use cases represent the functional requirements of the system from the perspective of its users.

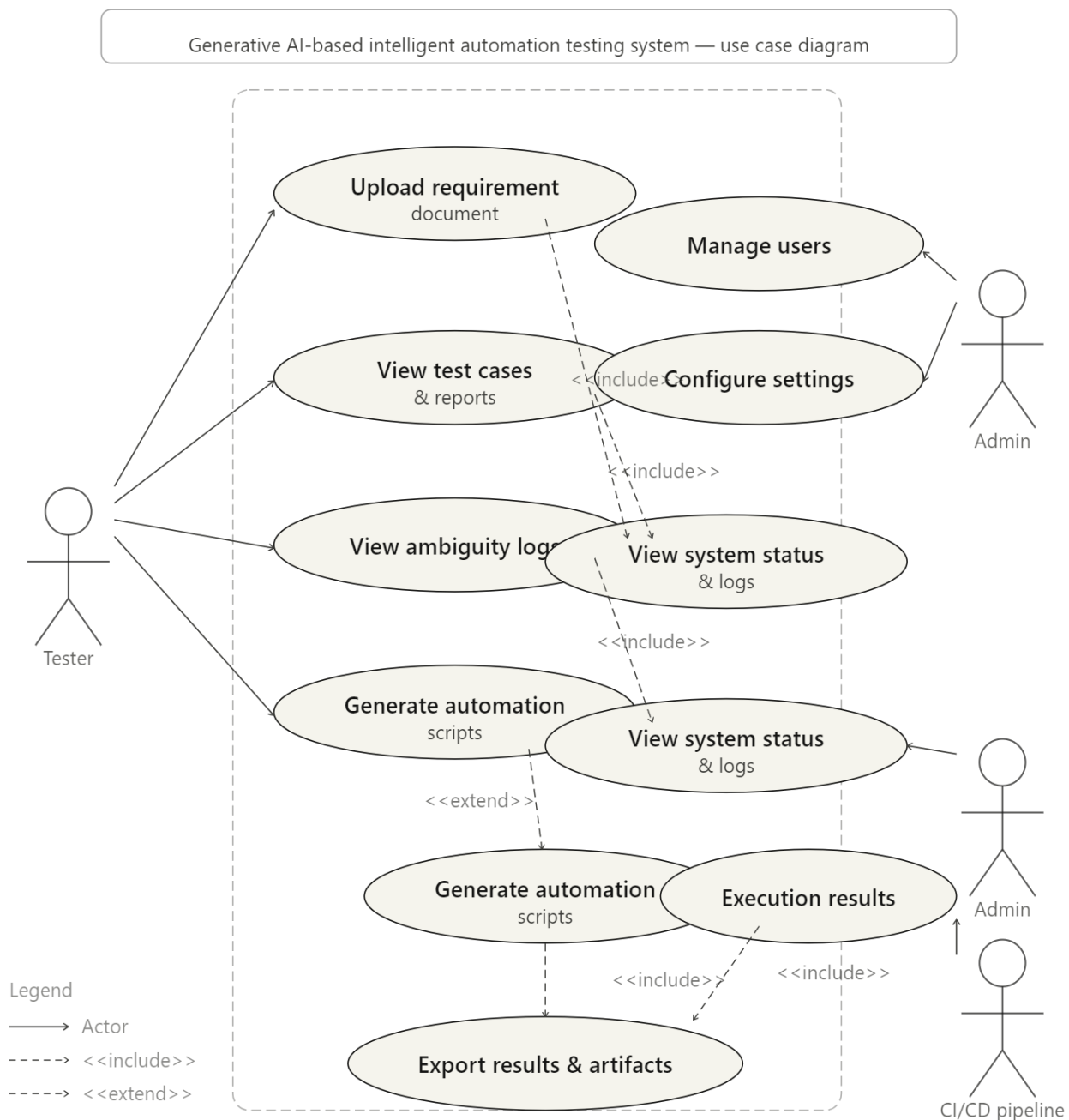


Fig. 4 Generative AI-based intelligent automation testing system -use case

5.6 Activity Diagram

Activity Diagrams model the dynamic behaviour of AI TestGen by showing the flow of activities and decisions within the system. They are equivalent to a detailed flowchart of the system's operational logic.

Activity Diagram (I): Input, Processing, and Finalization Flow

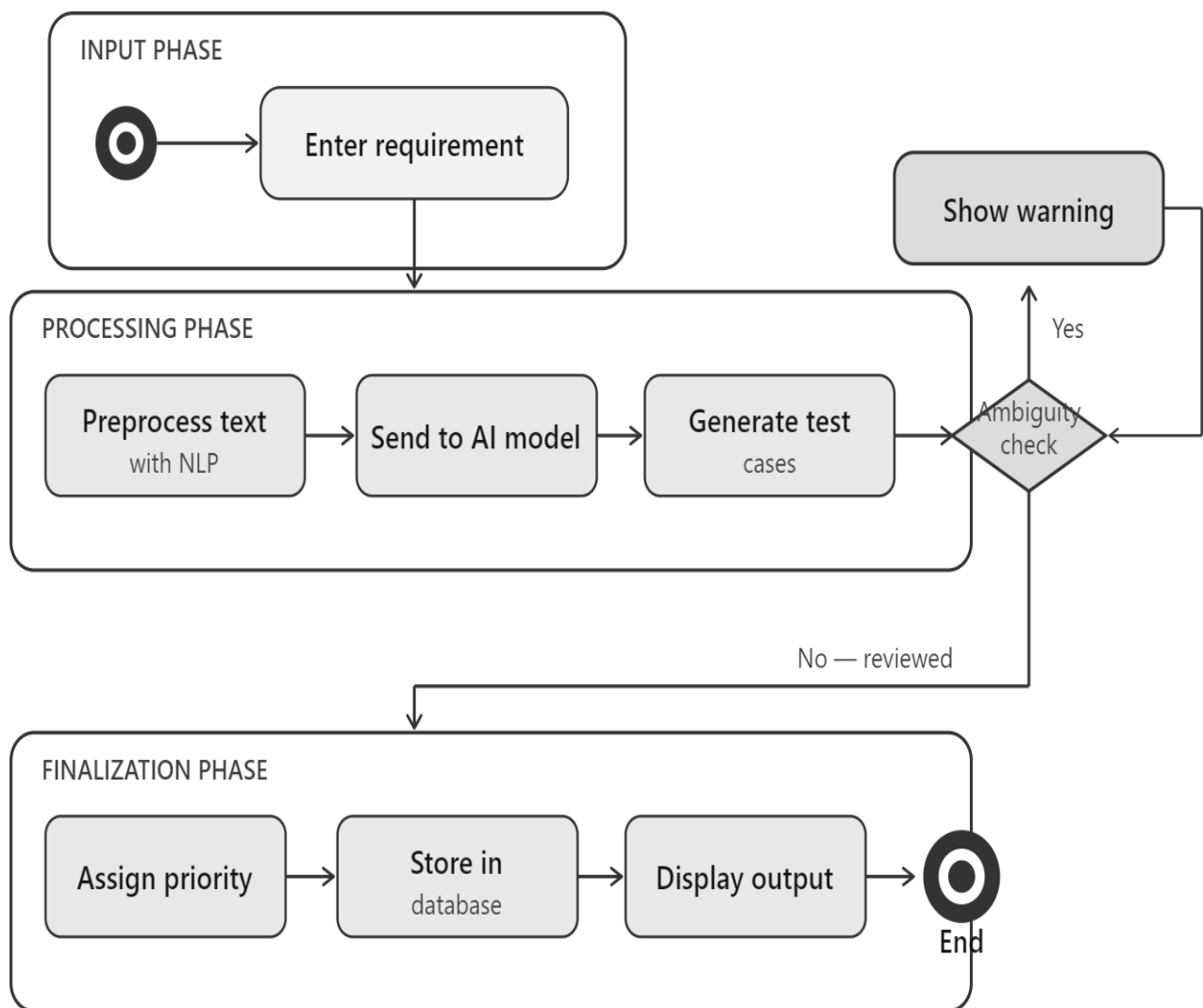


Fig. 5 Activity diagram : input processing and finalization flow

Activity Diagram (II): Overall System All Roles

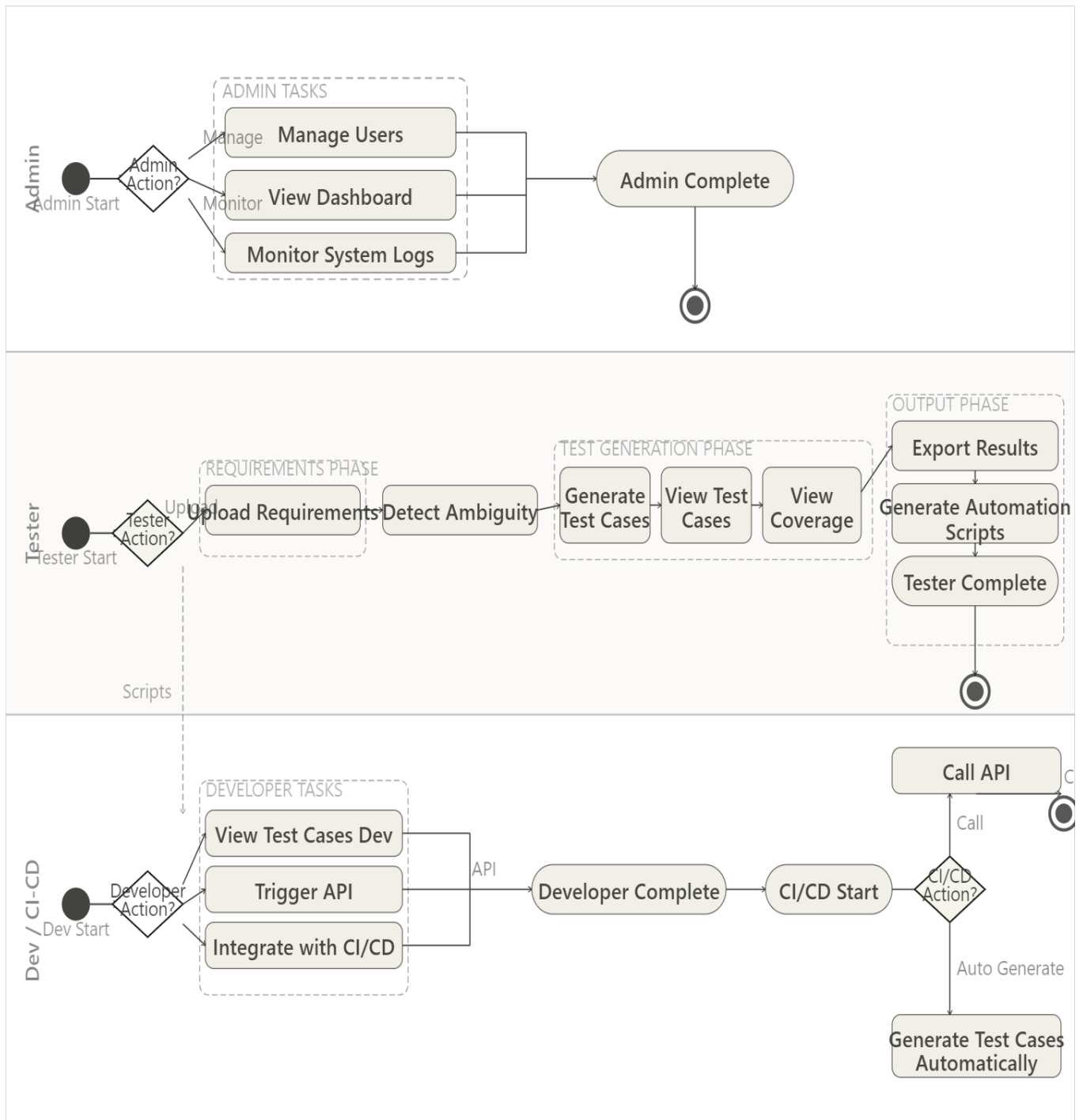


Fig. 6 Activity Diagram – Overall System All Roles

5.7 Sequence Diagram

The Sequence Diagram represents the time-ordered flow of messages between system components during the primary interaction: AI test generation flow from user input to stored test cases displayed on the dashboard.

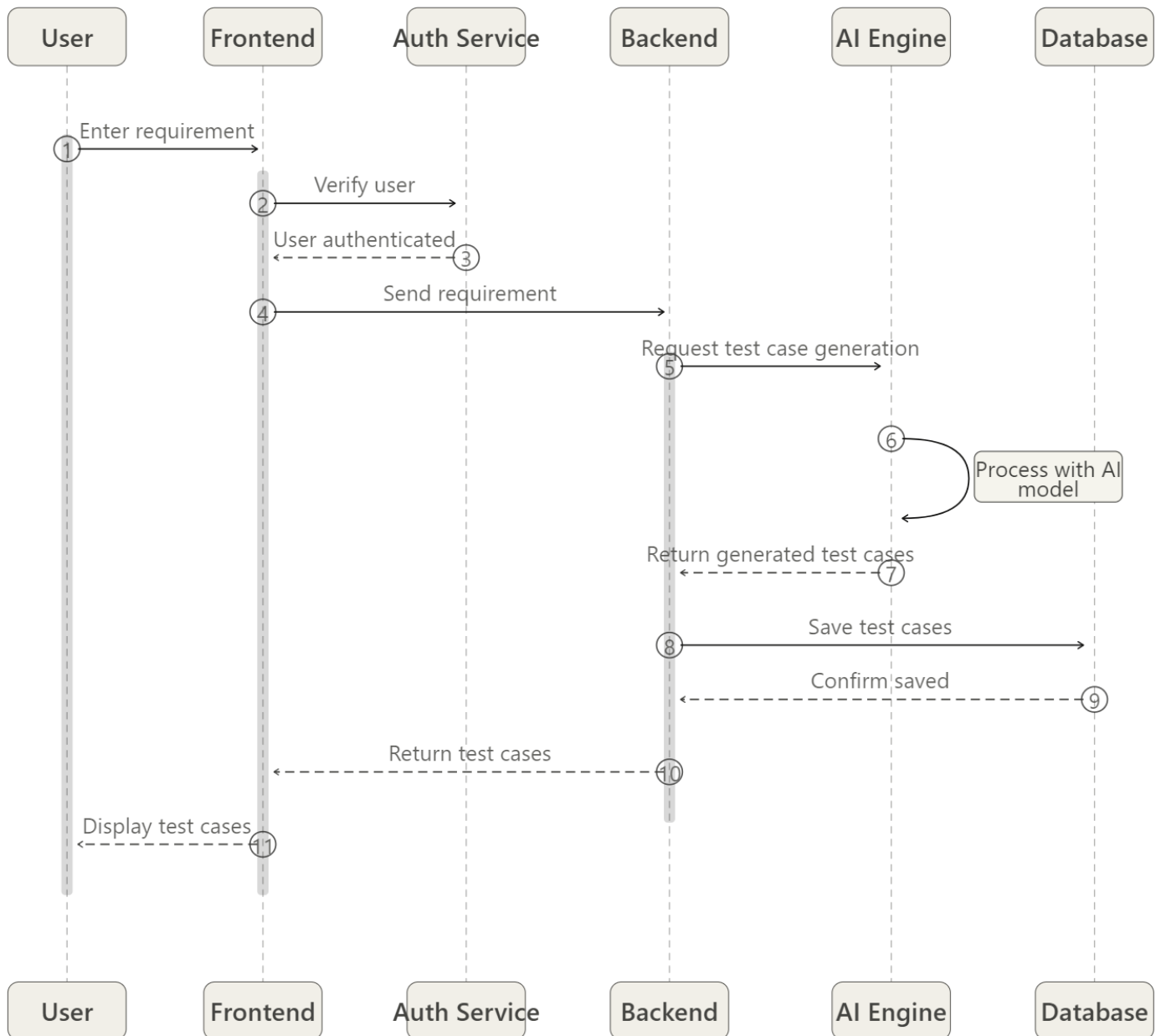


Fig. 7 Sequence Diagram – AI Test Generation Flow

5.8 Class Diagram

The Class Diagram models the static structure of AI TestGen's data layer and key service classes, showing the main entities (Users, Requirements, Test Cases, Coverage, AI Engine, Automation Scripts, Risk Analysis), their attributes, and the relationships between them.

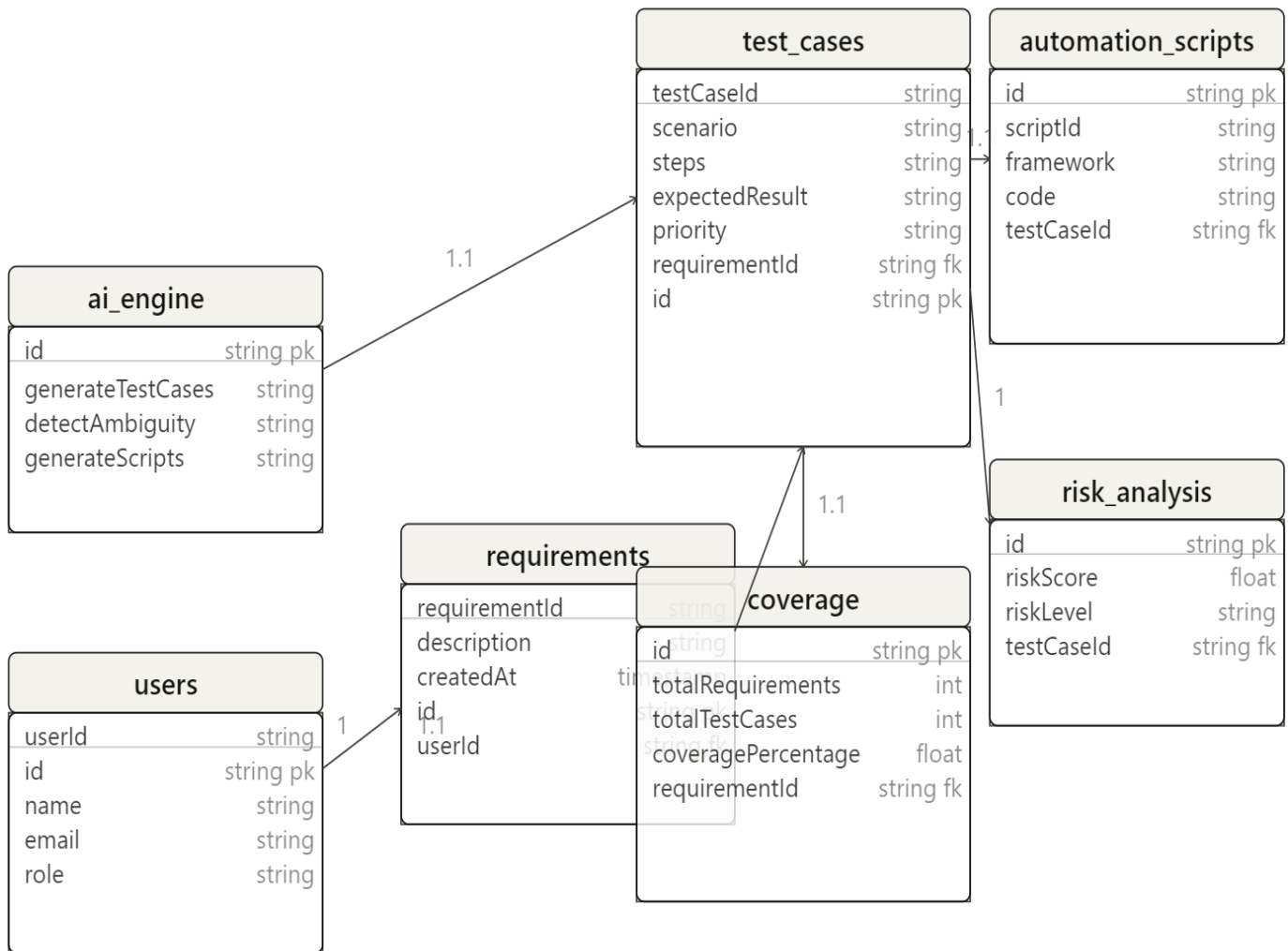


Fig. 8 Class Diagram – GenAI Testing System

Chapter 6

Project Plan

6.1 Weeks TimeLine

Week	Phase	Activities
Week 1	Planning	Project topic finalization, problem understanding, requirement gathering
Week 2	Planning	Literature review, research paper analysis, defining objectives
Week 3	Design	System architecture design, module identification
Week 4	Design	UI/UX wireframes, database schema design (MongoDB)
Week 5	Development	Setup project environment (React, Node.js, MongoDB)
Week 6	Development	Implement authentication (JWT, login/register)
Week 7	Development	Develop SRS input module & basic UI dashboard
Week 8	Development	Implement AI integration (aiService.js)
Week 9	Development	Test case generation module (core feature)
Week 10	Development	Ambiguity detection + prioritization module
Week 11	Development	Automation script generator + duplicate detection
Week 12	Development	Risk analysis + coverage + analytics dashboard
Week 13	Testing	Unit testing, API testing, bug fixing
Week 14	Testing	Integration testing, performance optimization
Week 15	Deployment	Final deployment setup, Docker/hosting configuration
Week 16	Documentation	Report writing, PPT preparation, final review

Table 3:Weeks Timeline

6.2 Weeks Timeline-Gantt Chart :

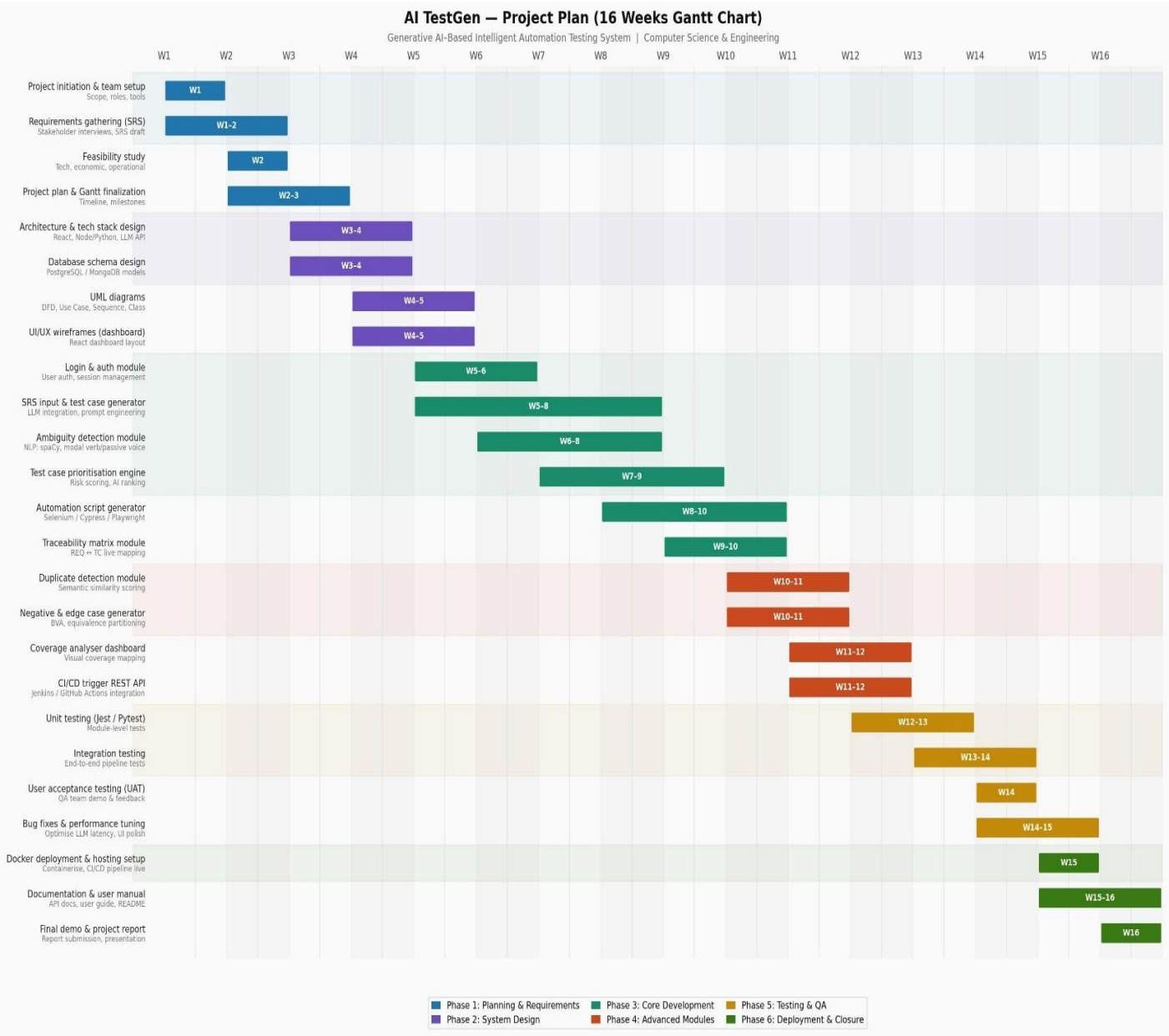


Fig 8.Gantt chart of Weeks Timeline

Chapter 7

Implementation / Mathematical Modeling

AI TestGen is organised into nine functional modules accessible through the React dashboard, each designed for a specific operational role in the intelligent automated testing workflow.

7.1 Modules Of AI TestGen

Module 1: Login / Access OS

The login screen provides secure authenticated access to the AI TestGen platform. Users enter their registered email address and password to access the Testing Meta-Verse dashboard. The interface displays the AI TestGen brand identity and system status indicator.

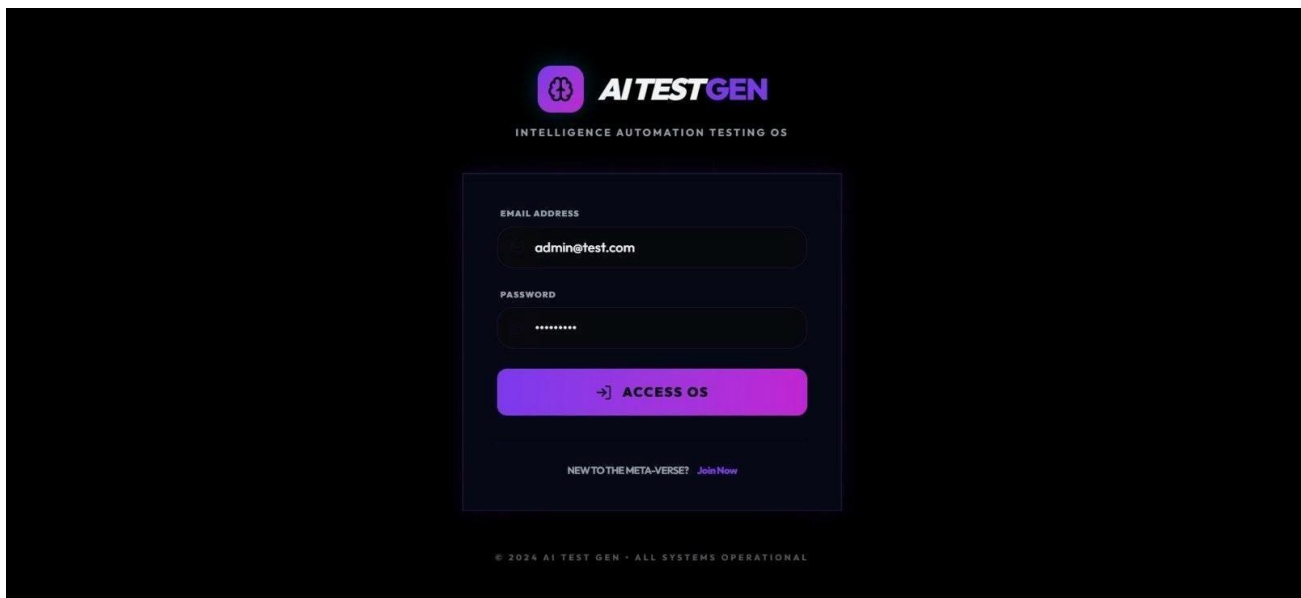


Figure 7.1: Login Module – Access os

Module 2: Dashboard

The main dashboard presents the Testing Meta-Verse welcome screen with global analytics KPI cards showing Created test cases, Detected ambiguities, Critical issues, and Overall coverage metrics. The left navigation panel provides access to all nine functional modules. The project selector at the top allows switching between multiple active projects.

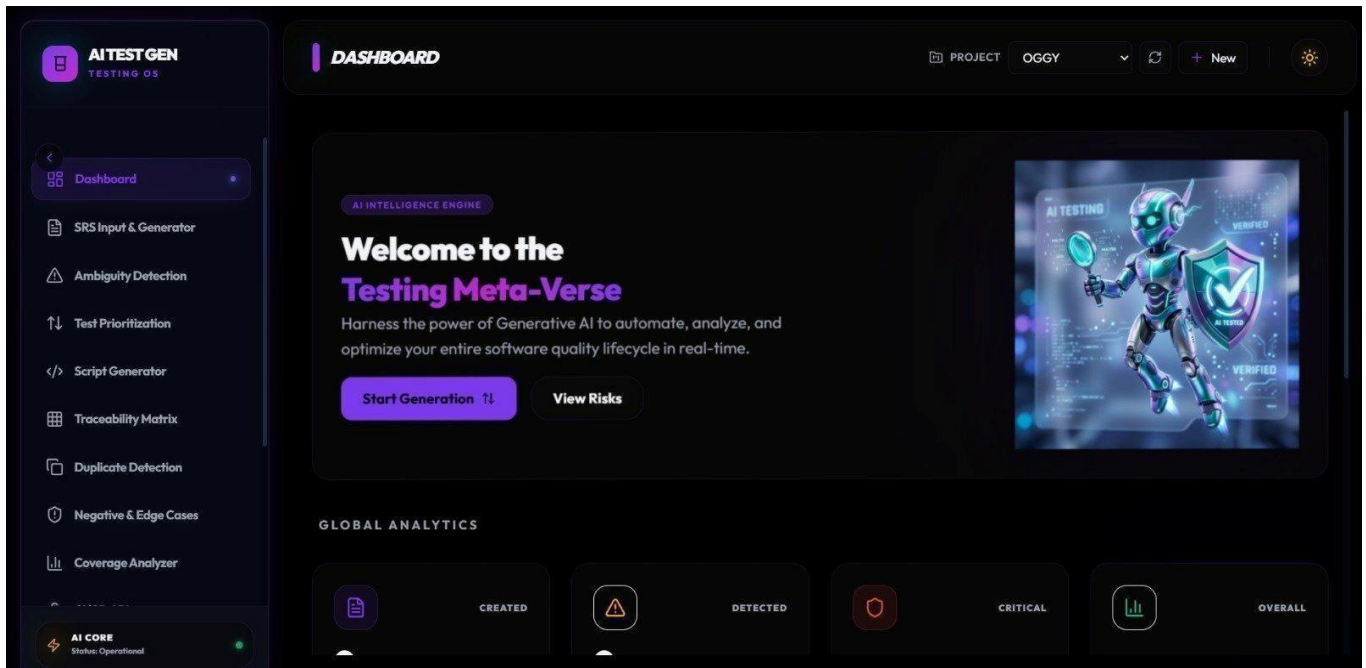
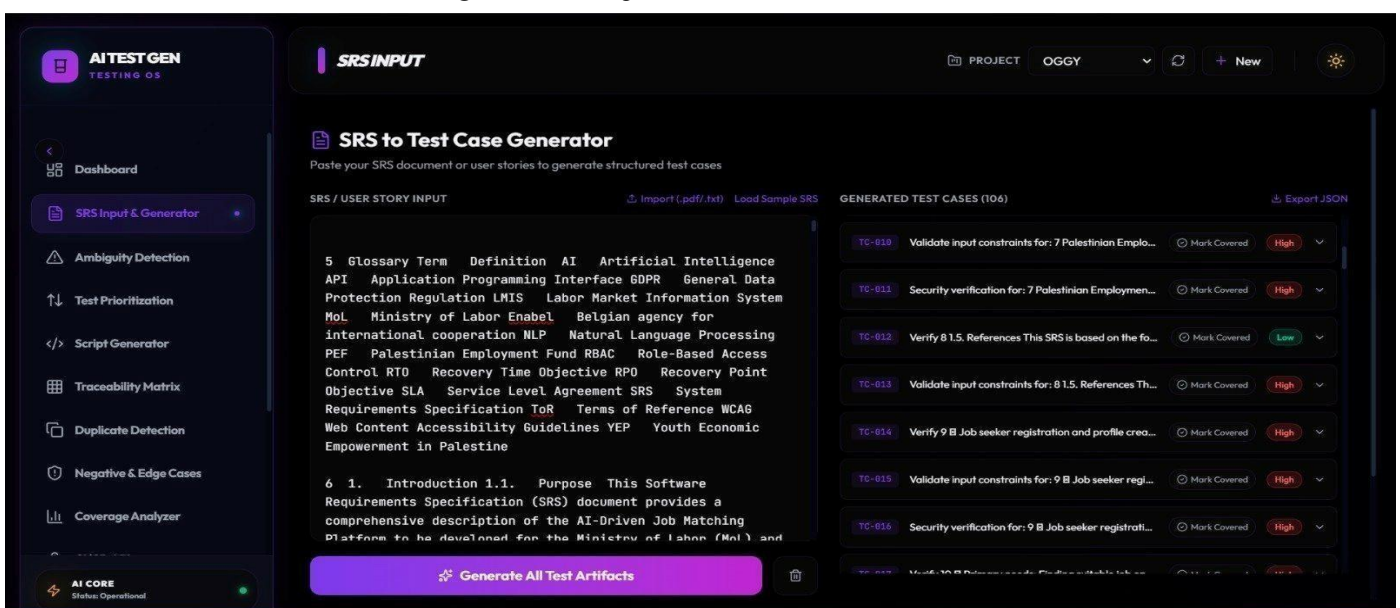


Figure 7.2: Dashboard Module – Testing Meta – Verse

Module 3: SRS Input & Generator

The SRS Input module provides a split-panel interface: the left panel accepts raw SRS text (or imported .pdf/.txt files) and the right panel displays the generated test cases in real time. The demonstration environment shows 106 test cases generated from a Palestinian Employment Fund job matching platform SRS, each tagged with a test case ID (TC-010, TC-011, etc.) and priority level (High/Low). The "Generate All Test Artefacts" button at the bottom triggers the full AI pipeline.

Figure 73: SRS Input & Generator Module



Module 4: Ambiguity Detection

The Ambiguity Detection module displays NLP-powered analysis results with severity counters: 6 High Severity, 22 Medium Severity, and 22 Low Severity issues (50 total) detected in the demonstration SRS. The left panel shows the annotated SRS text and the right panel lists each detected issue with its line reference, issue type, explanation, and a specific suggestion for clarification. Example issues include modal verb ambiguity ("may"), passive voice constructions, and vague terms ("standard", "generally").

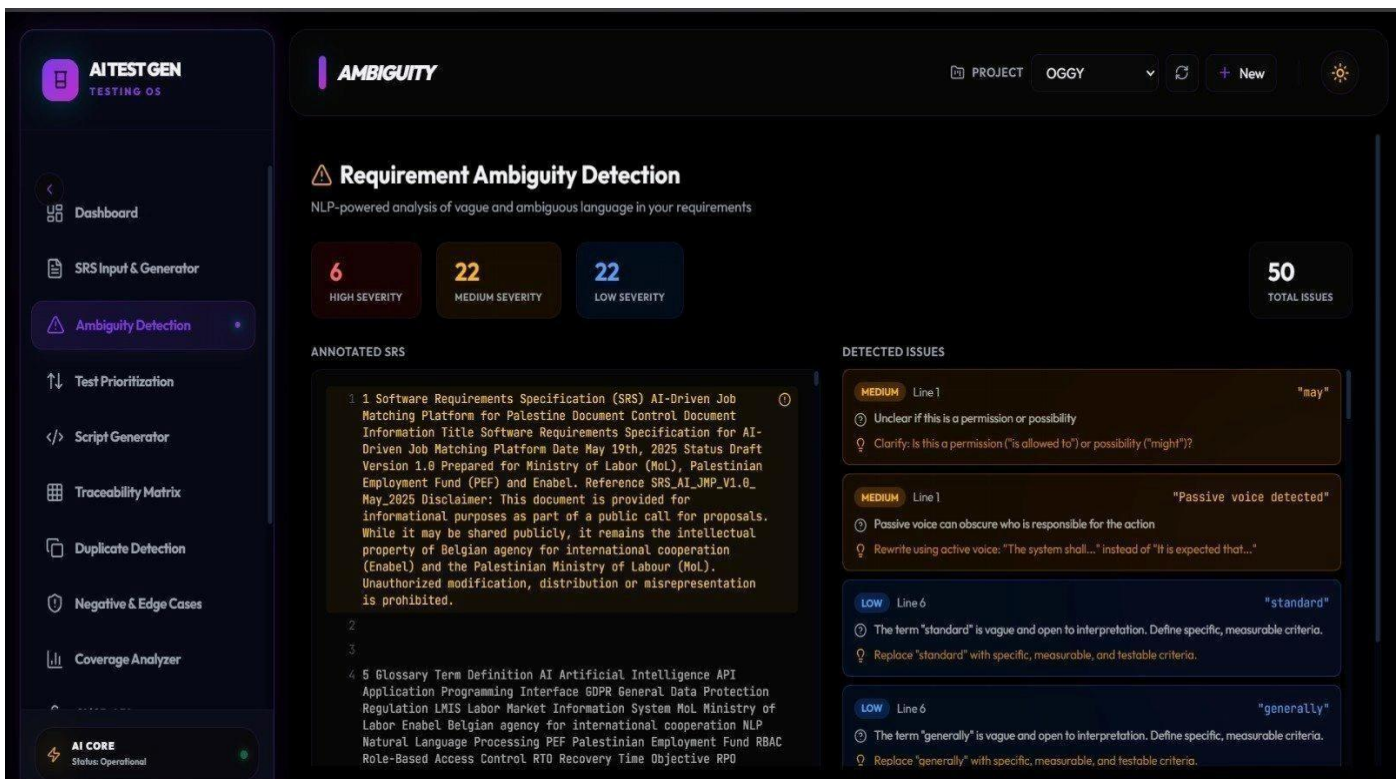


Figure 74: Ambiguity Detection Module

Module 5: Test Case Prioritisation

The Prioritisation module shows the AI-ranked test case table. Summary cards display 173 High Priority (74%), 47 Medium Priority (20%), and 13 Low Priority (6%) test cases. Each row shows the test case ID, scenario description, priority badge, risk score bar (colour-coded from red to green), category (Functional / Security), and linked requirement ID. The demonstration shows multiple test cases at risk score 100 for security-critical and functional requirements.

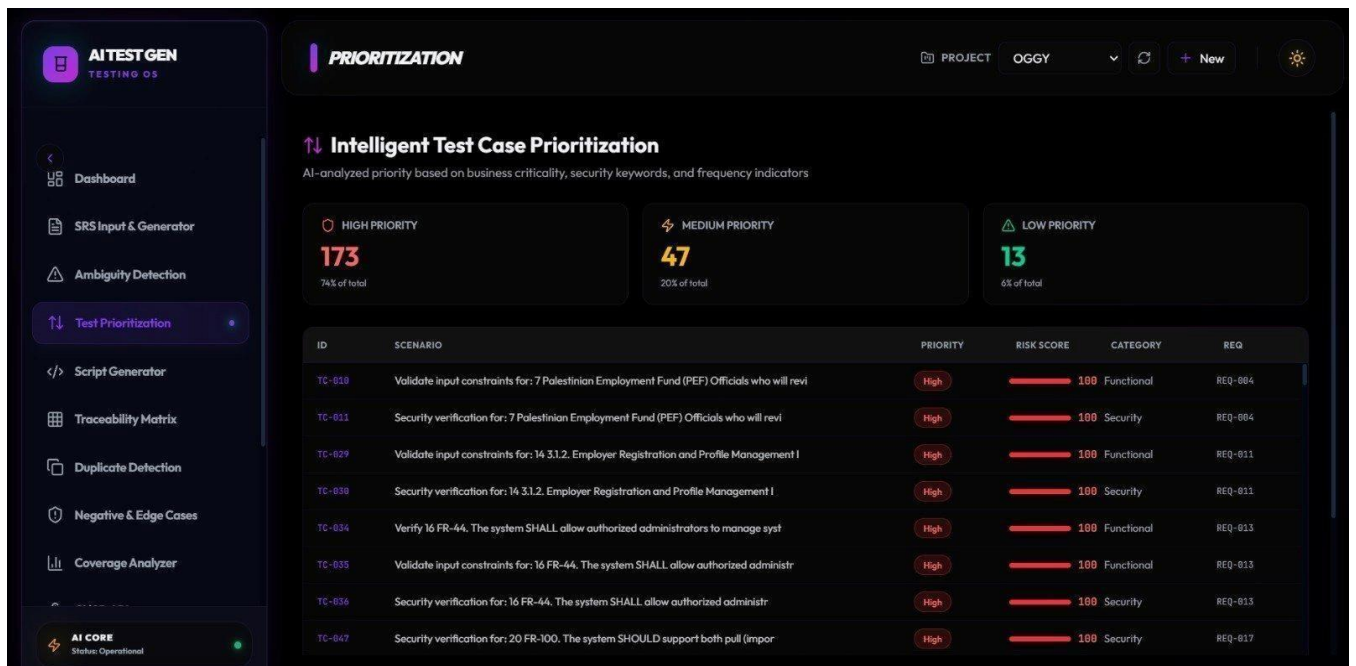


Figure 7.5: Test Case Prioritization Module

Chapter 8

Performance Evaluation and Testing

8.1 Performance Evaluation

The performance of the Generative AI–Based Intelligent Automation Testing System is evaluated based on the time complexity and efficiency of its core modules.

Test Case Generation: Time complexity depends on input size (SRS length).

Approximate complexity: $O(n)$, where n is the length of requirement text.

Ambiguity Detection: Uses NLP + keyword scanning $\rightarrow O(n)$ complexity.

Duplicate Test Case Detection:

Uses similarity comparison between test cases $\rightarrow O(n^2)$ in worst case.

8.2 Testing Techniques and Tools Used

Various types of testing were performed to ensure system reliability and performance:

1. Unit Testing

- Each backend module (controllers, services) was tested individually.
- Tool used: Jest

2. Integration Testing

- Verified interaction between frontend, backend, and AI modules.
- Ensured correct API responses.

3. Functional Testing

- Tested all features like: Test case generation , Ambiguity detection ,Script generation

4. API Testing

- Checked REST API endpoints using tools like: Postman

5. UI Testing

- Ensured proper rendering of dashboard, charts, and forms.

6. Performance Testing

- Checked response time of AI modules and database queries.

7. Security Testing

- Verified: JWT authentication ,Password encryption (bcrypt) ,Role-based access control

8.3 Test Case

Test Case No	Description	Input	Expected Output	Result
TC01	User Login	Valid username & password	Successful login	Pass
TC02	User Login	Invalid credentials	Error message	Pass
TC03	SRS Input	Valid requirement text	Stored successfully	Pass
TC04	Test Case Generation	SRS input	Generated test cases	Pass
TC05	Ambiguity Detection	Vague requirement	Highlight ambiguity	Pass
TC06	Test Prioritization	Generated test cases	Priority assigned	Pass
TC07	Script Generation	Test case input	Automation script generated	Pass
TC08	Duplicate Detection	Similar test cases	Duplicate flagged	Pass
TC09	Risk Analysis	Test case data	Risk score displayed	Pass
TC10	Export Feature	Test cases	CSV/JSON file download	Pass

Chapter 9

Result and Analysis

9.1 Proposed Work

AI TestGen proposes a comprehensive, multi-module intelligent test automation platform that addresses all the identified problems through LLM-powered test generation, NLP-based ambiguity detection, risk-based prioritisation, and automated script synthesis.

The proposed system is a web-based application that operates as follows:

1. **SRS Input & Test Case Generation:** When a requirements document or user story is submitted to the system (via text paste or .pdf/.txt upload), the LLM engine analyses the SRS and automatically generates a structured set of test cases covering functional verification, input constraint validation, and security checks. The system generated 106 test cases from a sample SRS in the demonstration environment.
2. **Requirement Ambiguity Detection:** The system performs NLP-powered analysis of the uploaded SRS, detecting vague terms, passive voice constructions, undefined modal verbs, and context-dependent language. Issues are categorised by severity (High, Medium, Low) and annotated inline with suggested clarifications.
3. **Intelligent Test Case Prioritisation:** All generated test cases are scored and ranked by the AI engine based on business criticality, security keyword indicators, and requirement frequency signals. Priority levels (High, Medium, Low) with numeric risk scores are assigned to guide testing effort allocation.
4. **Automation Script Generator:** For any selected test case, the system generates executable automation scripts in the target framework (Selenium, Cypress, or Playwright) with appropriate assertions, data inputs, and setup/teardown hooks.
5. **Traceability Matrix:** The system maintains a real-time requirements-to-test-case traceability matrix, enabling impact analysis for requirement changes and compliance reporting.
6. **Duplicate Detection:** The AI engine scans the generated test suite for semantically duplicate or near-duplicate test cases, flagging redundancies to reduce suite bloat.
7. **Negative & Edge Cases:** The system generates boundary value, equivalence partition, and error-path test cases for all identified requirements, ensuring systematic coverage of failure modes.

8. Coverage Analyser: A visual coverage dashboard maps test cases to requirements, highlighting uncovered or under-covered areas and computing an overall coverage percentage.
9. CI/CD Trigger API: A REST API endpoint allows external CI/CD systems (Jenkins, GitHub Actions, GitLab CI) to trigger test generation and retrieve generated test artefacts programmatically.

The following techniques and approaches are employed in the proposed system:

1. LLM-Powered Test Generation: The system uses a large language model with carefully engineered prompts to parse SRS text, identify testable conditions, and generate structured test cases in a consistent JSON/tabular format.
2. NLP Ambiguity Classification: Linguistic analysis using NLP techniques (modal verb detection, passive voice identification, vague term recognition) is applied to the SRS text, with severity scoring based on the impact of each ambiguity type on testability.

9.2 Comparison: Proposed System vs Existing System

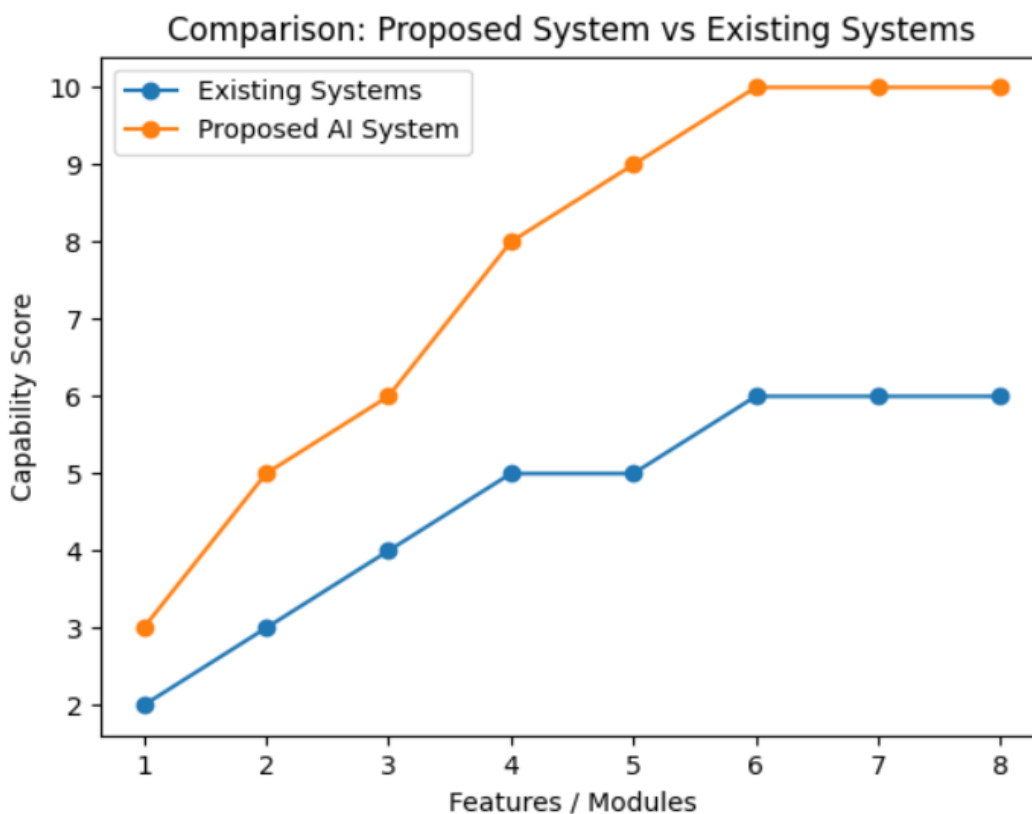


Fig 9.2 Comparison: Proposed System vs Existing System

The bar chart compares the key features of existing testing systems with the proposed Generative AI–Based Intelligent Automation Testing System. It clearly shows that existing systems have limited capabilities, mainly focusing on basic test case generation and lacking advanced functionalities. In contrast, the proposed system achieves higher scores across all features due to its integration of AI-driven modules. Features such as ambiguity detection, automation script generation, and risk analysis significantly enhance system performance. The chart highlights that the proposed system provides better scalability, accuracy, and automation compared to traditional approaches. Overall, it visually demonstrates that the proposed system is more efficient, intelligent, and suitable for modern software testing needs.

Application

- 1) Agile & DevOps Pipelines** — AI TestGen integrates directly with CI/CD tools like Jenkins and GitHub Actions, automatically generating and executing test cases on every code commit, enabling continuous quality assurance without manual intervention.
- 2) Enterprise Software Development** — Large organisations building complex ERP, CRM, or banking systems can use AI TestGen to instantly generate hundreds of structured test cases from lengthy SRS documents, drastically cutting QA timelines.
- 3) Healthcare & Regulated Industries** — In compliance-driven sectors like healthcare and finance, AI TestGen's traceability matrix ensures every requirement is mapped to a verified test case, supporting audit trails and regulatory reporting.
- 4) E-Commerce & Web Application Testing** — Teams building web platforms can use the script generator to automatically produce Selenium, Cypress, or Playwright scripts, enabling rapid regression testing across frequent UI updates and new feature releases.
- 5) Academic & Research Environments** — Computer Science departments and software engineering researchers can use AI TestGen as a practical demonstration platform for studying LLM-based test generation, NLP ambiguity detection, and AI-driven QA methodologies.

Conclusion

AI TestGen successfully demonstrates how Generative AI and Large Language Models can transform the software testing lifecycle by automating test case generation, ambiguity detection, risk prioritisation, and script synthesis from SRS documents. The system addresses the critical limitations of traditional manual testing methods by delivering structured, prioritised, and executable test artefacts in minutes rather than days, significantly reducing engineering effort and improving overall test coverage. With its versatile applications across Agile pipelines, enterprise development, regulated industries, and e-commerce platforms, AI TestGen proves to be a scalable and industry-ready solution. By integrating seamlessly with CI/CD workflows and supporting multiple automation frameworks, it empowers development teams to achieve faster release cycles and higher defect detection accuracy, establishing a strong foundation for the future of intelligent software quality assurance.

Future Prospects

AI TestGen can be extended in several important directions to further increase its value and applicability:

- 1)Test Management Tool Integration :** Integration with JIRA, TestRail, and Azure DevOps to automatically import generated test cases into existing project management and test tracking workflows.
- 2)Domain-Specific LLM Fine-Tuning :** Fine-tuning the LLM engine on domain-specific SRS datasets (banking, healthcare, e-commerce) to improve test quality, terminology accuracy, and coverage for specialised industry verticals.
- 3)Self-Healing Test Automation :** Implementing self-healing capabilities where the system detects UI or API changes in the target application and automatically regenerates or updates affected automation scripts without human intervention.
- 4)Natural Language Test Execution :** Integrating with AI agents capable of executing natural language test instructions directly against live applications, closing the loop from requirement to automated test result.
- 5)Mobile Application Testing Support :** Extending script generation to support Appium for native iOS and Android application testing, expanding the platform beyond web and API-based applications

References

- [1] C. Arora, M. Sabetzadeh, L. Briand, and F. Zimmer, “Automated Extraction and Clustering of Requirements Glossary Terms,” *IEEE Transactions on Software Engineering*, vol. 43, no. 10, pp. 918–945, 2015.
- [2] E. Paja, F. Dalpiaz, and P. Giorgini, “Modelling and Reasoning over Security Requirements in Socio-Technical Systems,” *Data & Knowledge Engineering*, vol. 115, pp. 167–184, 2018.
- [3] S. Yoo and M. Harman, “Regression Testing Minimisation, Selection and Prioritisation: A Survey,” *Software Testing, Verification and Reliability*, vol. 22, no. 2, pp. 67–120, 2012.
- [4] G. Fraser and A. Arcuri, “EvoSuite: Automatic Test Suite Generation for Object-Oriented Software,” in *Proceedings of the 19th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*, pp. 416–419, 2011.
- [5] T. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [6] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [7] M. Harman and B. F. Jones, “Search-Based Software Engineering,” *Information and Software Technology*, vol. 43, no. 14, pp. 833–839, 2001.
- [8] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York: McGraw-Hill Education, 2019.
- [9] IEEE Std 830-1998, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Standards Association, 1998.

Appendices

Plagiarism Report

